

Atividade: Problemas de outras disciplinas – resolução em múltiplas abordagens

Diogo Estéfano Cambraia Freitas | Engenharia de Telecomunicações | Unipampa

Boa tarde professor. Devido ao meu crescente interesse por física, optei por escolher a disciplina de Física I. Com isso, escolhi um problema baseado em tópicos visto em sala de aula recentemente (anotações logo abaixo).

EXPERIÊNCIA

1ª LEI: "Um corpo permanece em repouso ou em MUV até que uma força resultante atue sobre ele!"

2ª LEI: $\sum_i \vec{F}_i = m\vec{A}$

$\vec{F}_{\text{res}} \equiv \sum_i \vec{F}_i$

Se $\vec{F}_{\text{res}} = 0$, então $\vec{A} = 0$ e o corpo mantém seu estado.

1ª EXPERIÊNCIA

$\vec{MG}$ DECOMPÕE EM

- $MG \sin \theta \hat{i}$
- $MG \cos \theta \hat{j}$

DADO QUE

$\sum_i \vec{F}_i = 0$

$\sum_i \vec{F}_i = \vec{MG} + \vec{F}_{AT} + \vec{N} = 0$

$MG \sin \theta \hat{i} - MG \cos \theta \hat{j} - F_{AT} \hat{i} + F_N \hat{j} = 0$

CHEGA-SE EM

$MG \sin \theta \hat{i} - MG \cos \theta \hat{j} - \mu_e F_N \hat{i} + F_N \hat{j} = 0$

* COMO É EQUAÇÃO VETORIAL, TIRAMOS AS EQUAS

$\hat{i}: MG \sin \theta - \mu_e F_N = 0 \quad (A)$

$\hat{j}: -MG \cos \theta + F_N = 0 \quad (B)$

1) ISOLE F_N NA (B)

$F_N = MG \cos \theta$

2) SUBSTITUINDO NA (A)

$MG \sin \theta - \mu_e [MG \cos \theta] = 0$

III) $\sin \theta - \mu_e \cos \theta = 0$

IV) $\mu_e = \frac{\sin \theta}{\cos \theta} = \tan \theta$

$\vec{F}_{AT} = -F_{AT} \hat{i}$

$\vec{F}_N = N \hat{j}$

$\vec{F}_{AT} \perp \vec{F}_N$

$F_A = \mu_e N$

$\mu_e = \text{COEFICIENTE DE ATRITO ESTÁTICO}$

Problema

(a) Um bloco está inicialmente em repouso sobre um plano inclinado. O coeficiente de atrito estático é μ_s . Qual o ângulo crítico θ_t para que o bloco comece a deslizar?

(b) Pensei no seguinte código.

- 1- Definir as constantes para o problema ($g = 9.8$, $\mu_s = 0.35$, etc)
- 2- Definir as regras “físicas” da realidade (o bloco não se mexerá se o vetor de força gravitacional que o faz descer for $<$ que a força que o “segura”)
- 3- Testar usando loops vários valores de ângulo (que é o que precisa ser descoberto) até achar o valor certo (que condiz com as regras da “realidade”).

(c) Três implementações por LLM:

(I) Loop por “força bruta” (creio que a mais parecida com os passos que expliquei acima)

```
import math

# 1 - constantes
g = 9.8
mu_s = 0.35

# passo angular (precisão)
step = 0.001 # radianos (~0.057 graus)

# 2 - regra física (função de decisão)
def f(theta):
    # sin(theta) <= mu_s * cos(theta) -> repouso
    return math.sin(theta) - mu_s * math.cos(theta)

# 3 - teste de valores até encontrar transição
theta = 0
theta_max = math.pi / 2

while theta <= theta_max:
    if f(theta) > 0:
        break
    theta += step

print("Ângulo crítico (varredura):", math.degrees(theta), "graus")
```

Aqui nesta é realmente somente um loop While que fica iterando até achar o momento em que o ângulo θ_t faz o bloco cair.

(II) Solução analítica (com uma fórmula)

```
import math

# 1 - constantes
g = 9.8
mu_s = 0.35

# 2 - regra física (equilíbrio no limite)
#  $\sin(\theta) = \mu_s \cdot \cos(\theta)$ 
#  $\Rightarrow \tan(\theta) = \mu_s$ 

theta_c = math.atan(mu_s)

# 3 - resultado
print("Ângulo crítico (analítico):", math.degrees(theta_c), "graus")
```

Resolvendo de forma analítica (fazendo física mesmo) se chega ao fato de que:
 $\theta_t = \arctan(\mu_s)$
Logo, é uma simples conta. É só aplicar a função matemática e o resultado é o ângulo máximo que precisamos.

(III) Usando busca binária

```
import math

# 1 - constantes
g = 9.8
mu_s = 0.35

# 2 - regra física
def f(theta):
    return math.sin(theta) - mu_s * math.cos(theta)

# intervalo inicial
a = 0
b = math.pi / 2

# precisão desejada
tol = 1e-10

# 3 - busca do ângulo crítico
while (b - a) > tol:
    m = (a + b) / 2

    if f(m) > 0:
```

```
        b = m
    else:
        a = m

theta_c = (a + b) / 2

print("Ângulo crítico (bisseccção):", math.degrees(theta_c), "graus")
```

Não conhecia a busca binária. Pelo oque li, é como ir “cortando” metades, sabendo que o valor procurado está entre 2 pontos. Seria algo como, visto que o bloco deve começar a escorregar entre 0 e 90 graus, analisa o meio primeiro (45 graus), se ele não caiu ainda, daí pode aumentar a inclinação (óbvio). Com isto, vai para a nova metade desse novo intervalo. E assim se segue eliminando vários valores de uma vez. Até apertar o valor procurado. Achei bem interessante e otimizado. Tentarei aplicar em códigos futuros.